Lecture 7: Assembly II — Control Flow, Stack & Procedures

Session 8 · Wed Sep 23, 2026

Contents

Run It: `lea` Builds Small Multiples . . . 2
When One `lea` Is Not Enough . . . 2
Why Use `lea` to Multiply? . . . 2
Two Shifts Right, and Signedness Matters . . . 3
The Flags Have Been Waiting . . . 3
Where We Are . . . 3
A Jump Is a Write to `rip` . . . 4
A Loop, Byte by Byte . . . 4
How a Jump Stores Its Target . . . 4
The Kinds of Jump . . . 4
Compare, Then Jump . . . 5
What the Four Flags Mean . . . 5
The Jump You Pick Says Signed or Unsigned . . . 6
Worked Example: One Compare, Two Answers . . . 6
Run It: Same Bits, Two Answers . . . 6
If-Else Is a Jump Over Code . . . 6
Sometimes There Is No Jump At All . . . 7
`cmov` Step by Step . . . 7
Run It: The Compiler Decides, Not Your Source . . . 7
When `cmov` Is Not Allowed . . . 8
Loops Are a Jump Backwards . . . 8
Run It: `while` and `for` Are the Same Code . . . 8
Now the Stack . . . 9
Where the Stack Lives . . . 9
`push` and `pop` . . . 9
`call` and `ret` . . . 10
Run It: Print the Return Address . . . 10
The Stack Frame . . . 11
Who Saves What . . . 11
Run It: A Function Keeping Its Promises . . . 11
Leaf Functions Get a Shortcut . . . 12
Recursion Is One Frame Per Call . . . 12
Run It: Watch the Stack March Down . . . 12
Something Extra in That Listing . . . 12
AI Systems Connection . . . 13
Run It: Warp Divergence on a Real GPU . . . 13
The Lecture in One Slide . . . 13
Practice . . . 13
Practice — Answers 1–3 . . . 14
Practice — Answers 4–5 . . . 14
Practice — Answers 6–7 . . . 14

Reading 14

Run It: `lea` Builds Small Multiples

Back to Lecture 6. Each line below is a **separate function**, and each function is just one `leaq` followed by `retq`:

```
long times2(long x) { return x * 2; }
long times3(long x) { return x * 3; }
long times5(long x) { return x * 5; }
long times9(long x) { return x * 9; }
long affine(long x) { return 8 * x + 7; }
```

```
times2:  leaq  (%rdi,%rdi),   %rax    # x + x  = 2x
times3:  leaq  (%rdi,%rdi,2), %rax    # x + 2x = 3x
times5:  leaq  (%rdi,%rdi,4), %rax    # x + 4x = 5x
times9:  leaq  (%rdi,%rdi,8), %rax    # x + 8x = 9x
affine:  leaq  7(,%rdi,8),    %rax    # 8x + 7
```

`leaq D(B,I,S)` computes `D + B + I` × `S` and puts the number in the register. It never reads memory. With `x` in both `B` and `I`, one `leaq` gives `x + S·x`, and `S` can only be 1, 2, 4 or 8.

When One `lea` Is Not Enough

For other multipliers the compiler combines steps. Real `clang -O2` output:

```
times7:   leaq  (,%rdi,8), %rax       # 8x
          subq  %rdi, %rax            # 8x - x = 7x

times11:  leaq  (%rdi,%rdi,4), %rax   # 5x
          leaq  (%rdi,%rax,2), %rax   # x + 2·(5x) = 11x

times100: imulq $100, %rdi, %rax      # too many steps: just multiply
```

The compiler uses `lea` only while it is cheaper than `imul`: up to about two simple instructions. Past that, it multiplies.

Why Use `lea` to Multiply?

It is **faster**, but only by a couple of CPU cycles:

Instruction	Latency (cycles until the result is ready)
simple `leaq`, e.g. `(%rdi,%rdi,2)`	1
`imulq`	3

- **Lower latency.** `3x` is ready 2 cycles sooner. That only adds up in a hot loop where each step waits for the one before.
- **More hardware can run it.** `imul` runs on one execution unit; `lea` can run on several, so several `lea`s can run at the same time.
- **It does not touch the flags.** `lea` never writes `RFLAGS`, so the compiler can put it between a `cmp` and its jump without changing where the jump goes.
- **It is not always smaller.** `leaq (,%rdi,8), %rax` is 8 bytes, because an address with no base register has to carry a 4-byte zero.

You do not need to write this yourself. You need to **recognize it**: `lea` with no memory access is usually a multiply.

Two Shifts Right, and Signedness Matters

Shifting right by `n` divides by 2^n (by 16 when `n` is 4). x86-64 has **two** right shifts, and they differ in what they fill in at the top:

```
shrb  $4, %al      # logical:    fills with zeros     -> unsigned
sarb  $4, %al      # arithmetic: fills with the sign  -> signed
```

Example: the byte `0xF0` shifted right by 4.

```
          bits        as unsigned             as signed
start     1111 0000   240                     -16
shrb $4   0000 1111    15  right: 240/16       15  wrong: should be -1
sarb $4   1111 1111   255  wrong: should be 15  -1  right: -16/16
```

The compiler picks the shift from the **C type**. `x >> 1` compiles to `shrl %eax` for an `unsigned` and to `sarl %eax` for an `int` (no count means shift by 1). With `x = -8`: `sarl` gives `-4`, but `shrl` gives `2147483644`.

The Flags Have Been Waiting

Lecture 6 ended with an instruction whose result we never used:

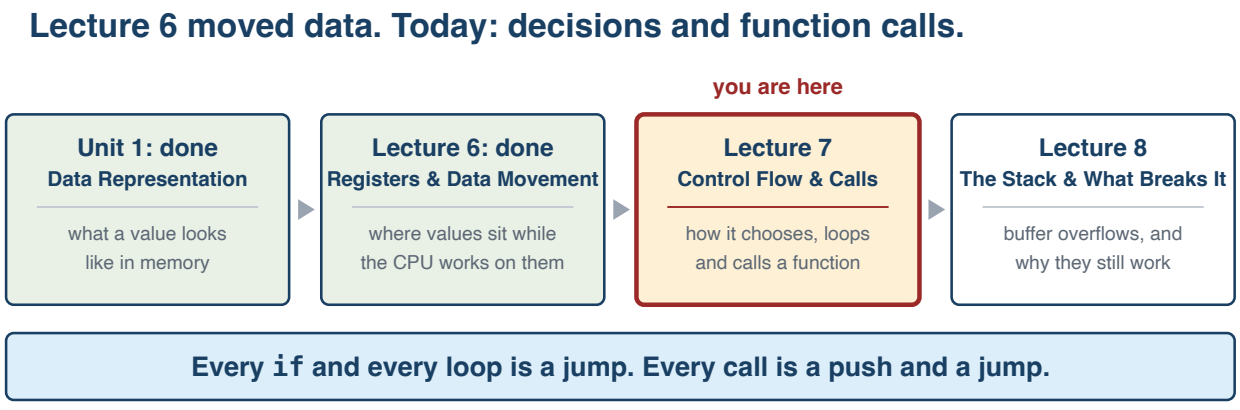
```
cmpq %rcx, %rsi     # computes rsi - rcx, sets ZF, SF, CF, OF
```

`cmpq` subtracts `%rcx` from `%rsi`, throws the answer away, and keeps four **flag** bits that describe the answer. Today, jump instructions read those flags to decide which instruction runs next. That is how `if` and loops work.

Where the flags live: in a special register called **`RFLAGS`** (`gdb` calls it `eflags`). Each flag is one bit of it: CF is bit 0, ZF bit 6, SF bit 7 and OF bit 11.

- It is not one of the 16 general registers, so you cannot `mov` into or out of it. Arithmetic instructions write it; jumps, `set` and `cmov` read it.
- Each CPU core has its own. When the OS switches programs, it saves and restores `RFLAGS` along with the other registers.
- In gdb, `info registers eflags` prints something like `eflags 0x246 [ PF ZF IF ]`: the flags that are currently 1.

Where We Are



Today has two parts. First, **jumps**: how the CPU decides which code runs next. Second, the **stack**: how one function calls another and gets back. Lecture 8 shows how the stack can be attacked.

A Jump Is a Write to `rip`

`rip` is the **instruction pointer**: a 64-bit register that holds the address of the next instruction the CPU will run.

Normally, after each instruction the CPU adds that instruction's length to `rip`, so execution moves straight down the listing. An x86-64 instruction is between 1 and 15 bytes long, so `rip` moves by a different amount each time.

A jump puts a different address into `rip`. The next instruction then comes from that address instead:

```
jmp .somewhere      # rip = address of .somewhere, unconditionally
```

- `.somewhere` is a **label**: a name that the assembler turns into an address.
- You cannot write to `rip` with `mov`. Only jumps, `call` and `ret` change it.
- The machine has **no loop instruction**. A loop is a jump back to an address the program has already run.

A Loop, Byte by Byte

This loop counts `%ecx` down from 3 to 0. The left column is each instruction's address; the middle column is its bytes:

```
0:  b9 03 00 00 00    movl  $3, %ecx    # 5 bytes
5:  ff c9             decl  %ecx        # 2 bytes; ZF = 1 once ecx is 0
7:  75 fc             jne   5           # 2 bytes; jumps to 5 if ZF = 0
9:  c3                retq
```

Follow `rip` one instruction at a time:

```
rip   instruction      ecx  ZF   next rip
0x0   movl $3, %ecx     3   –    0x5    0x0 + 5 bytes
0x5   decl %ecx         2   0    0x7
0x7   jne  5            2   0    0x5    jump taken: back to the top
0x5   decl %ecx         1   0    0x7
0x7   jne  5            1   0    0x5    jump taken
0x5   decl %ecx         0   1    0x7
0x7   jne  5            0   1    0x9    not taken: go on to retq at 0x9
```

How a Jump Stores Its Target

The bytes `75 fc` do not contain the address `5`. They contain a **distance**:

- `75` means "`jne`, followed by a 1-byte distance".
- `fc` is that distance as a signed byte: `0xfc` is −4.
- The distance is counted from the **next** instruction. That is at `0x9`, and `0x9 - 4 = 0x5`.

This is a **relative** jump. It still works if the whole function is loaded at a different address, because the distance to the target does not change.

Form	Size	Reaches	Real encoding
short `jmp`	2 bytes	−128 to +127 bytes	`eb 00`
long `jmp`	5 bytes	about ±2 GB	`e9 2c 01 00 00`
long `jne`	6 bytes	about ±2 GB	`0f 85 c9 fe ff ff`

The assembler picks the shortest form that reaches the label. `objdump` prints the target address it worked out (`jne 0x5 <top>`), not the raw distance.

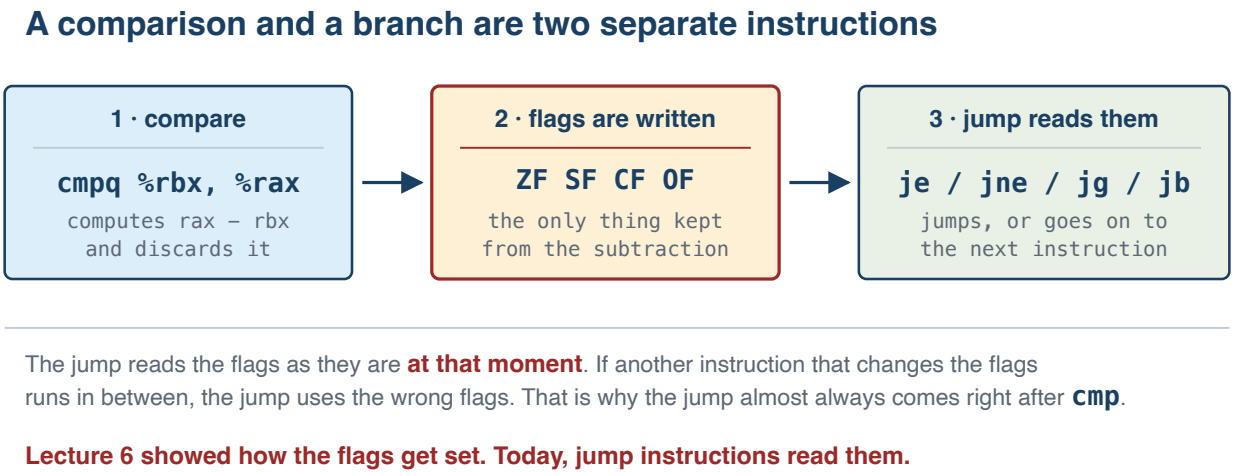
The Kinds of Jump

Kind	AT&T form	The new `rip` comes from	Used for
direct	`jmp .L2`	a label, fixed when the program is built	loops, `else`
conditional	`je .L2`	the label **if** the flags say so, else the next instruction	`if`, loops
indirect, register	`jmp *%rax`	the value in `%rax`	`switch` (gcc)
indirect, memory	`jmp *(%rax)`	8 bytes read from memory	`switch` (CS:APP)

The `*` marks an **indirect** jump: "jump to the address stored here". A direct jump's target is fixed in the machine code; an indirect jump's target is only known when the program runs. A `switch` compiles to a **jump table**: an array of addresses (or offsets), one per `case`, and an indirect jump into it.

`call` and `ret` also change `rip`. They are covered in the second half.

Compare, Then Jump



A decision takes two instructions. First, `cmp` subtracts the two values and sets the flags. Then a jump such as `je` or `jl` reads the flags and either jumps or goes on to the next line. The jump reads the flags as they are at that moment, so if another instruction that changes the flags (such as `addq`) runs in between, the jump uses the wrong flags.

Mind the order: `cmpq %rbx, %rax` computes `rax - rbx`, the *second* operand minus the first.

What the Four Flags Mean

The CPU sets the flags automatically after every arithmetic or logic instruction: `add`, `sub`, `cmp`, `test`, `and`, `or`, `xor`. `mov`, `lea`, `push`, `pop` and `jmp` do not change them.

Flag	Set to 1 when the result…	Read by
ZF (zero)	is 0	`je`, `jne`
SF (sign)	has its top bit set (negative, if signed)	`js`, and the signed jumps
CF (carry)	does not fit as an **unsigned** number (for `cmp`: a borrow happened)	`jb`, `ja` (unsigned)
OF (overflow)	does not fit as a **signed** number	`jl`, `jg` (signed)

Conditional jumps, `set` instructions and `cmov` read the flags. Nothing else does.

The Jump You Pick Says Signed or Unsigned

Jump	Reads	Means
`je` / `jz`	ZF=1	equal / zero
`jne` / `jnz`	ZF=0	not equal / not zero
`jl jle jg jge`	SF, OF, ZF	**signed** less / greater
`jb jbe ja jae`	CF, ZF	**unsigned** below / above
`js` / `jns`	SF	result was negative / was not

`cmp` sets all the flags at once. The **jump** decides whether the values are treated as signed (`jl`, "less") or unsigned (`jb`, "below").

> **Warning**
>
> This is the signed/unsigned difference from Lecture 5, seen in assembly. `jl` and `jb` read different flags, so they can give different answers for the same bits. Mixing them up is a common mistake when reading compiler output, and using the wrong one in your own code is a real bug.

Worked Example: One Compare, Two Answers

Take `edi = -1` and `esi = 1`:

```
cmpl %esi, %edi      # computes edi - esi = -1 - 1 = -2
```

Flag	Value	Why
ZF	0	-2 is not zero
SF	1	-2 is negative: its top bit is 1
CF	0	as unsigned, $4294967295 - 1$ needs no borrow
OF	0	-2 fits in a signed 32-bit `int`

- `jl` jumps when SF $\neq$ OF. Here $1 \neq 0$, so it **jumps**: as signed, $-1 < 1$ is true.
- `jb` jumps when CF $= 1$. Here CF $= 0$, so it **does not jump**: as unsigned, $4294967295 < 1$ is false.

Run It: Same Bits, Two Answers

```
int  min  (int x, int y)          { return x < y ? x : y; }
unsigned min_u(unsigned x, unsigned y) { return x < y ? x : y; }
```

```
_min:     cmpl   %esi, %edi     _min_u:   cmpl   %esi, %edi
          cmovll %edi, %eax               cmovbl %edi, %eax
```

```
as signed:     a = -1,  b = 1     ->  a < b   true    (jl / cmovll)
same bits, unsigned:
  ua = 4294967295, ub = 1         ->  ua < ub false   (jb / cmovbl)
```

The `cmp` is the same and the bits are the same. The only difference is the letter after `cmov` (a conditional move, explained in two slides): `l` for signed, `b` for unsigned. The compiler picked it from the **types in your C**.

If-Else Is a Jump Over Code

```
if (x < y) return x;
else       return y;
```

```
    cmpl  %esi, %edi     # computes edi - esi, i.e. x - y
    jge   .else          # if NOT (x < y), skip the then-branch
    movl  %edi, %eax
    retq
.else:
    movl  %esi, %eax
    retq
```

The jump tests the **opposite** of the C condition. The C code says "if x < y, run this". The assembly says "if x >= y, jump past this". Expect every `if` you read in assembly to be flipped this way.

Sometimes There Is No Jump At All

Why a jump can be slow. The CPU works on many instructions at once, like an assembly line: it starts the next instructions before the current ones finish. At a conditional jump the flags are often not ready yet, so the CPU **guesses** which way the jump will go and keeps working down that path.

- **Right guess:** the jump costs almost nothing.
- **Wrong guess:** the CPU throws away the work it started on the wrong path and restarts at the right address. That costs about **15 to 20 cycles**.
- A loop's backward jump is easy to guess: it is taken every time but the last. An `if` on random data is guessed wrong about half the time.

The alternative. The compiler can compute both possible answers and pick one with a **conditional move** (`cmov`). There is no jump, so nothing to guess:

```
movl   %esi, %eax    # assume the answer is y
cmpl   %esi, %edi    # compute x - y, set the flags
cmovll %edi, %eax    # if x < y, replace it with x
```

`cmov` Step by Step

`cmovll` reads as `cmov` + `l` (**l**ess, the condition) + `l` (**l**ong, 32 bits). The condition letters are the same as for jumps: `cmove`, `cmovne`, `cmovl`, `cmovg`, `cmovb`, `cmova`, …

Trace `min(x, y)` for two inputs:

```
                    x = 3, y = 5              x = 7, y = 5
movl   %esi, %eax   eax = 5                   eax = 5
cmpl   %esi, %edi   3 - 5 = -2: SF=1, OF=0    7 - 5 = 2: SF=0, OF=0
cmovll %edi, %eax   SF≠OF, less: eax = 3      SF=OF, not less: eax stays 5
result              3                         5
```

- The destination must be a **register**. The source can be a register or memory, but not a constant: `cmovll $5, %eax` does not assemble.
- There is no byte version. `cmov` works on 16, 32 or 64 bits only.
- A memory source may be read **even when the condition is false**. That is why `cmov` cannot protect `*p` from a NULL pointer (see "When `cmov` Is Not Allowed").

Run It: The Compiler Decides, Not Your Source

```
int as_if     (int x, int y) { if (x < y) return x; else return y; }
int as_ternary(int x, int y) { return x < y ? x : y; }
```

```
_as_if:                  _as_ternary:
    movl   %esi, %eax        movl   %esi, %eax
    cmpl   %esi, %edi        cmpl   %esi, %edi
    cmovll %edi, %eax        cmovll %edi, %eax
    retq                     retq
```

The two functions compile to **exactly the same instructions**. Writing `?:` does not force a `cmov`, and writing `if` does not force a jump. The compiler chooses, based on whether it is safe to compute both sides (next slide).

When `cmov` Is Not Allowed

`cmov` computes **both** sides before it picks one. So the compiler can only use it when computing both sides is safe:

```
int guarded(const int *p) { return p ? *p : 0; }
```

```
_guarded:
    testq %rdi, %rdi
    je    .null        # a real branch -- *p would fault on NULL
    movl  (%rdi), %eax
    retq
```

If `p` is NULL, reading `*p` crashes, so the compiler must jump around the read. Side effects are the same: in `if (x < 0) count++;` the compiler cannot add to `count` and then undo it, so it must jump. **What decides is whether both sides are safe to run, not how you wrote the C.**

Loops Are a Jump Backwards

```
while (i < n) { total += a[i]; i++; }
```

```
    testq %rsi, %rsi        # n <= 0?
    jle   .empty            #   then skip the whole loop
.loop:
    addq  (%rdi,%rcx,8), %rax
    incq  %rcx
    cmpq  %rcx, %rsi        # rsi - rcx, i.e. n vs i
    jne   .loop             # backwards -- this is the loop
```

The compiler checks the condition once before the loop. After that, the body ends with a jump back to the top. Your `while` became **a check before the loop, plus a do-while**, so each pass through the loop needs one jump instead of two.

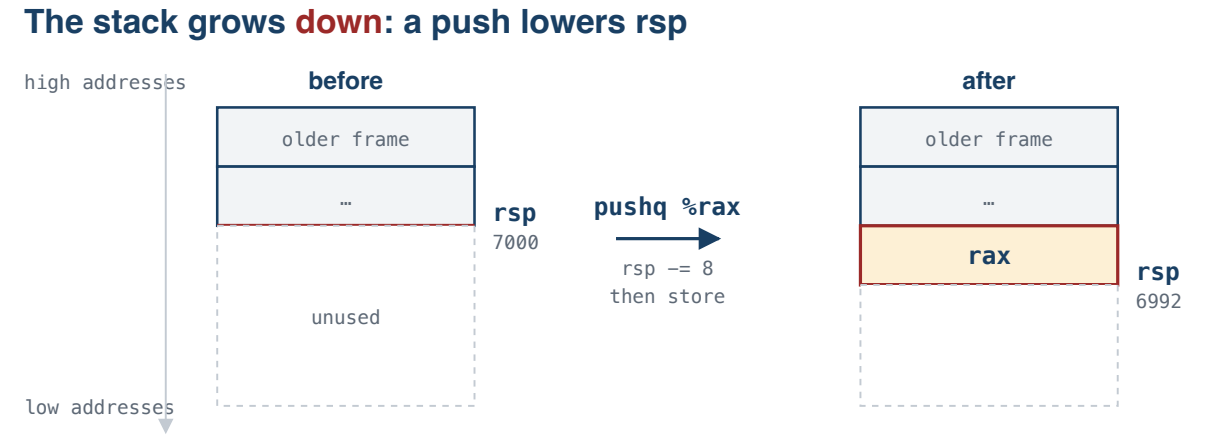
Run It: `while` and `for` Are the Same Code

```
sum_while   testq / jle / xorl / xorl / .loop: addq, incq, cmpq, jne
sum_for     testq / jle / xorl / xorl / .loop: addq, incq, cmpq, jne
```

The `while` version and the `for` version compile to exactly the same instructions. A `for` loop is a `while` loop written in a different order.

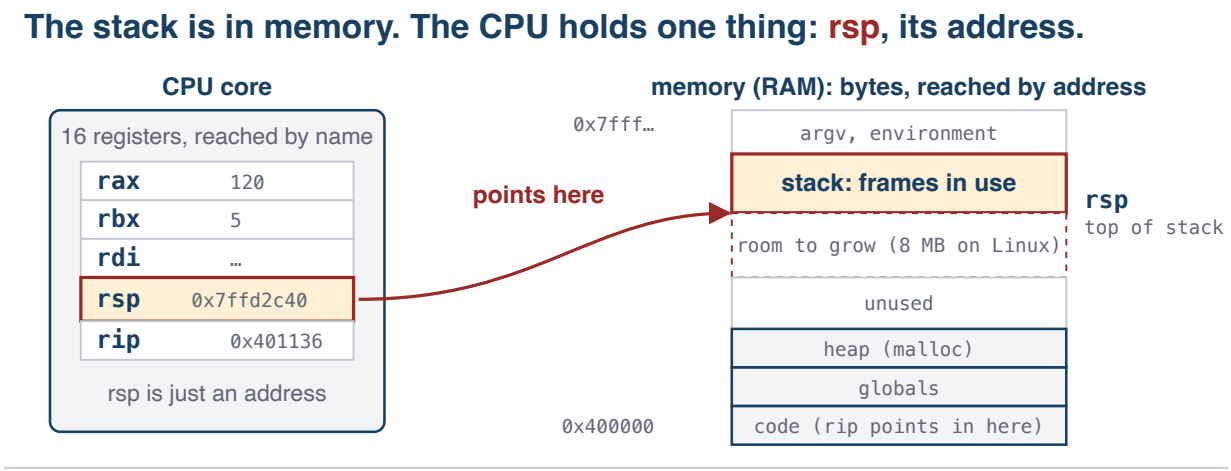
`sum_do` is different. A `do-while` loop always runs at least once, so it has no check before the loop.

Now the Stack



The stack is ordinary memory, and `rsp` holds the address of the last value put on it. The key fact: the stack grows **downward**. Each new value goes at a lower address, so "the top of the stack" is the **lowest** address in use.

Where the Stack Lives



The stack is ordinary memory, reserved by the OS when the program starts.

The CPU's entire share of the stack is one register: `rsp`. It holds one address, the lowest stack byte in use. Every value "on the stack" is a byte in memory reached through that address. `push` and `pop` change `rsp`; `call` and `ret` push and pop through it. The stack is a slice of memory. `rsp` says where.

`push` and `pop`

`push` puts one 8-byte value on the stack. `pop` takes the most recent one back off. Each is two steps, and the order matters.

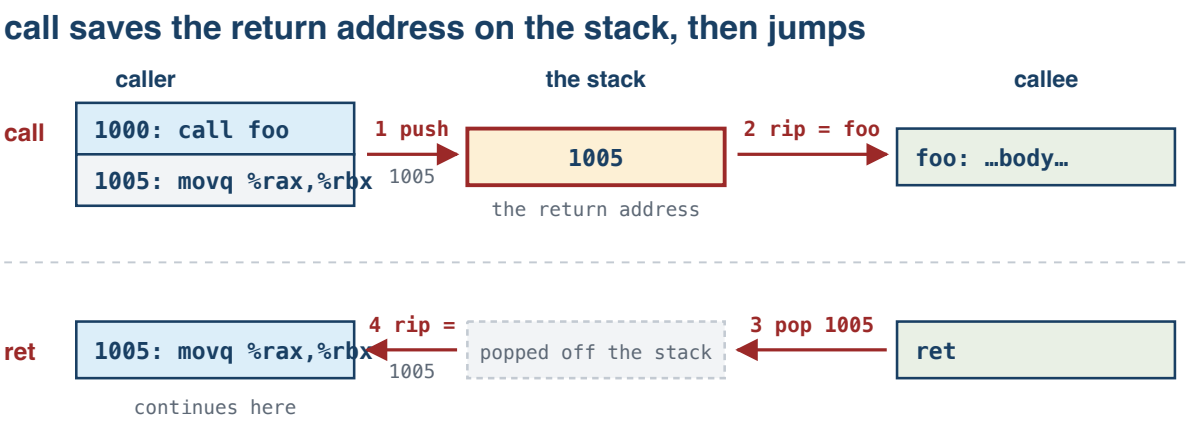
```
pushq %rax      # 1. rsp = rsp - 8       move rsp down to make room
                # 2. (%rsp) = rax        store rax in that room

popq  %rbx      # 1. rbx = (%rsp)        read the value rsp points at
                # 2. rsp = rsp + 8       move rsp back up, room is free
```

`pop` undoes `push`: the value comes out, and `rsp` returns to where it was.

There is no stack hardware behind this. `pushq %rax` is exactly `subq $8, %rsp` followed by `movq %rax, (%rsp)`, an ordinary subtract and an ordinary store. The CPU has a `push` instruction only because this pair is so common: it takes 1 byte of machine code instead of 8.

`call` and `ret`



`call` = push the address of the next instruction, then jump. **`ret`** = pop that address into **`rip`**.

The return address is ordinary memory that the program can overwrite. Lecture 8 is about that.

```
call foo      # push the address of the NEXT instruction, then jmp foo
ret           # pop that address back into rip
```

`call` is a push and a jump; `ret` is a pop into `rip`. It works because each function leaves the stack as it found it.

Run It: Print the Return Address

```
printf("%p\n", __builtin_return_address(0));
```

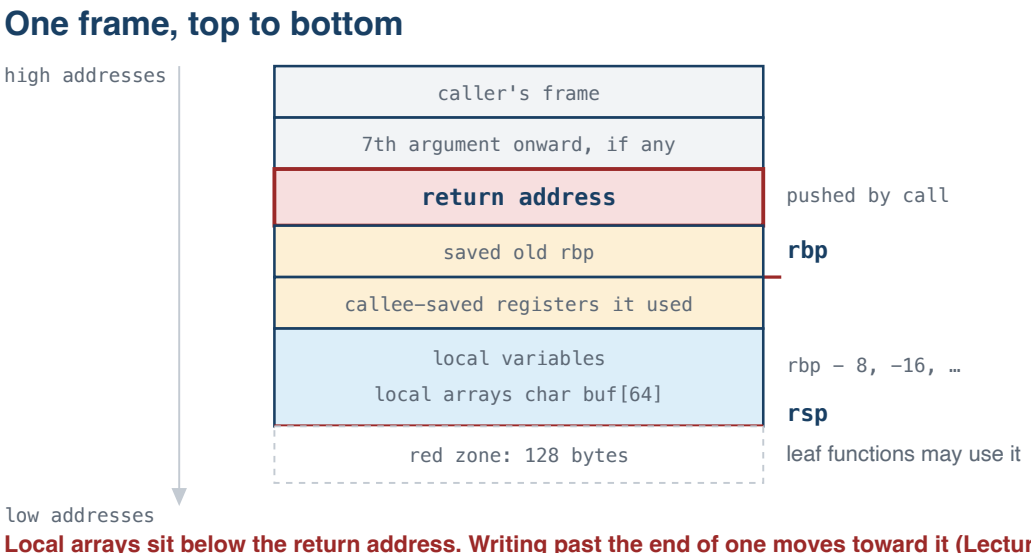
```
  outer returns to      0x104764488
  inner returns to      0x1047644ec

  a local in main is at 0x16b69a7c8
```

The return address is **8 ordinary bytes of memory that the program can write to**. It sits on the same stack as your local variables. Nothing marks it as special, and nothing checks it before `ret` uses it.

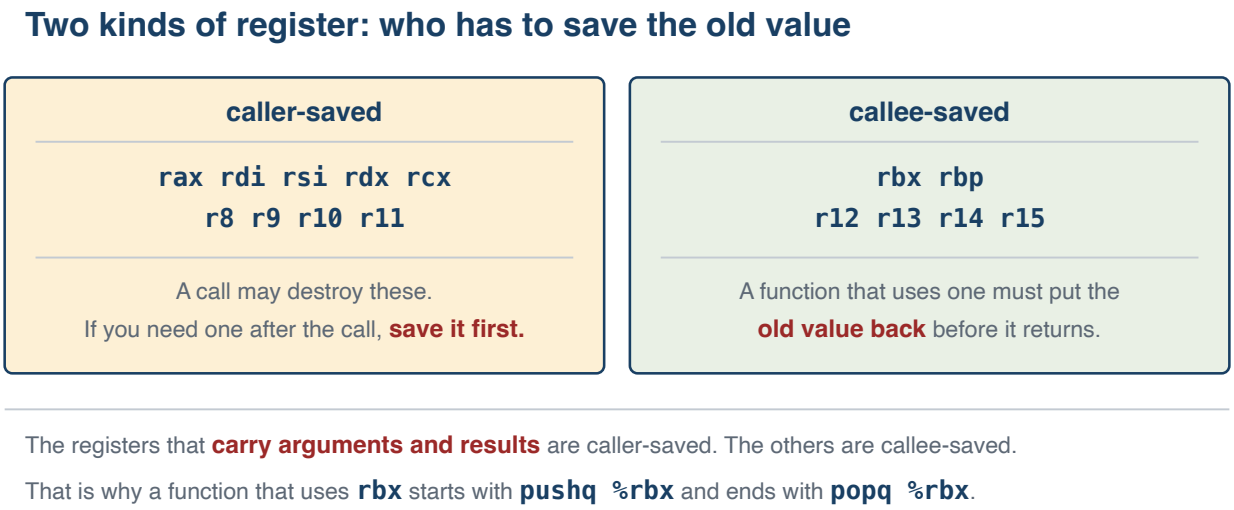
Remember this. Lecture 8 is built on it.

The Stack Frame



Each function call gets its own block of stack memory like this, called a **frame**. Look at where the **local arrays** are: below the return address. Writing an array from index 0 upward moves **toward the return address**.

Who Saves What



Caller-saved registers may be overwritten by any call, so save them yourself if you need them afterwards. **Callee-saved** registers must be put back by any function that uses them. Get this backwards and values change behind your back, in a way that looks like a compiler bug but is not.

Run It: A Function Keeping Its Promises

`n` is used after the recursive call, so the compiler keeps it in `rbx`. That register is callee-saved, so `fact` saves it first and restores it last:

```
static long fact(long n, int depth) {
    if (n <= 1) return 1;
    return n * fact(n - 1, depth + 1);     // n is used AFTER the call
}
```

```
_fact:
    pushq %rbp              # save the caller's rbp (depth will live there)
    pushq %rbx              # save the caller's rbx
    movq  %rdi, %rbx        # n -> rbx
    ...
    callq _fact             # recurse: rbx still holds n when this returns
    imulq %rbx, %rax        # n * fact(n - 1)
    ...
    popq  %rbx              # restore, in reverse order of the pushes
    popq  %rbp
    retq                    # every rbx/rbp/r12–r15 user ends this way
```

Leaf Functions Get a Shortcut

A function that does not call any other function is a **leaf**. It may use the 128 bytes just below `rsp`, called the **red zone**, without moving `rsp`. This is safe because no other function will run and overwrite that memory.

That is why `add` from Lecture 6 had no setup code at all: two instructions, no frame, no pushes. Most small functions look like that.

Recursion Is One Frame Per Call

The hardware does not know what recursion is. When `fact(5)` calls `fact(4)`, it is an ordinary `call`: a new return address, a new frame and new locals:

```
static long fact(long n, int depth) {
    volatile char pad[32];
    printf("depth %d  frame at %p\n", depth, &pad[0]);
    return n <= 1 ? 1 : n * fact(n - 1, depth + 1);
}
```

Run It: Watch the Stack March Down

```
  depth 0   frame at 0x16b31e748   n = 5
  depth 1   frame at 0x16b31e6c8   n = 4
  depth 2   frame at 0x16b31e648   n = 3
  depth 3   frame at 0x16b31e5c8   n = 2
```

Each frame is `0x80` bytes lower than the one before, because every call uses a frame of the same size. As the calls return, the frames are removed in reverse order, and each `ret` finds its return address where its `call` put it.

With a large enough argument, the frames run past the end of the stack's memory. That is a **stack overflow**, and this output shows it building up.

Something Extra in That Listing

The real `fact` had instructions we skipped:

```
    movq ___stack_chk_guard@GOTPCREL(%rip), %rax
    movq (%rax), %rax
    movq %rax, 32(%rsp)        # a known value, below the return address
    ...
    cmpq 32(%rsp), %rcx        # still there?
    jne  __stack_chk_fail      # if not, abort
```

The compiler added a **canary**: a known value stored between the local variables and the return address. Before returning, the function checks that the value has not changed. The compiler added it because `fact` has a local **array**.

It is there to catch a specific attack. Lecture 8 shows what that attack is.

AI Systems Connection

Branches cost more on a GPU than on a CPU. A GPU runs threads in groups of 32 that all execute the same instruction together. If an `if` sends some threads one way and the rest the other way, the GPU runs one path while the other threads wait, then runs the other path. Both paths run, one after the other. This is called *warp divergence*, and it is why avoiding branches matters even more on a GPU. The next slide measures it.

The calling convention is a real boundary. Which registers hold the arguments, how the stack is used, and who saves which registers: Python relies on these rules every time it calls compiled code, such as a PyTorch kernel. If a library was built for different rules, this is where it breaks, usually as an error when the library loads.

Run It: Warp Divergence on a Real GPU

The same `if` and the same work per thread. Only the way threads are split changes:

```
if ((i / 32) % 2 == 0) path_a(); else path_b();  // warp_aligned
if (i % 2 == 0)        path_a(); else path_b();  // divergent
```

```
NVIDIA GeForce RTX 4070 Laptop GPU, warp size 32, 4194304 threads

  no_branch        8.44 ms   every thread runs path A
  warp_aligned     9.28 ms   half the warps run A, half run B
  divergent       18.12 ms   every warp runs A, then B
```

In `divergent`, every warp holds both even and odd threads, so every warp runs path A and then path B. The same work takes **twice as long**.

The Lecture in One Slide

	what it is	the thing to remember
`jmp`	a write to `rip`	a loop is a jump back to earlier code
`jl` vs `jb`	signed vs unsigned	`cmp` sets the flags; the jump picks the meaning
`cmov`	a `mov` that runs only if the flags say so	used only when both sides are safe
a loop	a check, then a jump back	`while` becomes a check plus a do-while
`push` / `pop`	`rsp` down, `rsp` up	the stack grows **down**
`call` / `ret`	push and pop of the return address	the return address is ordinary memory
callee-saved	`rbx`, `rbp`, `r12`–`r15`	if you use one, put it back

Next lecture: **The Stack & What Breaks It**. What happens when a program writes past the end of a local array and reaches the return address.

Practice

1. Why does the assembly for `if (x < y)` jump on `jge` instead of `jl`?
2. Give a C fragment the compiler must compile to a real branch, and say why `cmov` is not allowed there.
3. `cmpl %ebx, %eax` followed by `ja`: is this comparing signed or unsigned values? What C types would produce it?

4. Write out what `call foo` does, in terms of `pushq` and `jmp`. Then write `ret` in terms of `popq`.
5. A function's first instruction is `pushq %rbx`. What does that tell you about what the function is going to do?
6. In `05-recursion.c`, each frame is `0x80` bytes lower than the last. Why would that number change if you enlarged `pad[]`?
7. `while (n > 0) { ... n--; }`: why must this use `jle` and not `jz`? What goes wrong with a negative `n`?

Practice — Answers 1–3

1. The jump's job is to **skip** the body, so it tests the opposite condition. The body runs when `x < y`; the jump leaves when `x >= y`:

```
cmpl  %esi, %edi      # x - y, flags only
jge   .L2             # x >= y: skip the body
```

2. `p ? *p : 0`. When `p` is NULL, reading `*p` crashes. `cmov` computes both sides before it picks one, so it would read `*p` anyway. A function call (`x ? f() : 0`) is the same: once `f` has run, it cannot be taken back.

3. Unsigned. `a` ("above") and `b` ("below") are the unsigned jumps; `g` and `l` are the signed ones. The `l` suffix means 32-bit operands, so the C is `unsigned int` (or `uint32_t`). A `size_t` or pointer would be `cmpq`.

Practice — Answers 4–5

4. `call` is a push and a jump. `ret` is a pop into `%rip`:

```
call foo    # pushq <address of the next instruction>
            # jmp   foo
ret         # popq  %rip   (not a real instruction, but it is what ret does)
```

5. `%rbx` is **callee-saved**. Saving it first means the function is about to overwrite `%rbx` and has promised to put it back. Expect a `popq %rbx` just before the `ret`. It usually means a value has to survive a call the function makes, since `%rbx` is not destroyed by calls.

Practice — Answers 6–7

6. A frame holds the locals, the saved registers and the return address, rounded up so `%rsp` stays a multiple of 16. `pad[]` is a local, so a bigger `pad` means a bigger frame. With `pad[64]` the gap grows from `0x80` to `0xa0`. It moves in 16-byte steps, not byte by byte.

7. `jz` only leaves when `n` is **exactly** 0. Start at `n = -3` and the loop should not run at all. With `jz` it runs, and `n--` moves *away* from zero, all the way down to `INT_MIN`, where signed overflow is undefined behavior. `jle` is a signed test, so it leaves for any `n <= 0`, which is what `n > 0` means:

```
testl %edi, %edi      # flags from n
jle   .Ldone          # n not positive: never enter
```

Reading

- CS:APP §3.6 (Control: condition codes, jumps, loops, conditional moves)
- CS:APP §3.7 (Procedures: the stack, calls, register-saving conventions)

CS:APP and these slides both use **AT&T** syntax: source first, destination last.